

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE:ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

*I **Mathiprakash E 192312085** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Math', enclosed within a circular flourish.

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

Sol. A perceptron should converge when the training data is linearly separable. If it does not, the problem is usually due to an implementation or training-parameter error.

Possible causes:

1. **Improper learning rate:** A very large learning rate can cause unstable weight updates, while an extremely small value makes convergence unnecessarily slow.
2. **Incorrect weight initialization:** Unusual or improperly assigned weights can make debugging difficult and may interact badly with other implementation errors.
3. **Bias handling error:** If the bias term is omitted or updated incorrectly, the decision boundary may not be positioned correctly.
4. **Incorrect update rule:** The standard update is:

$$w_i = w_i + \eta(y - y^{\wedge})x_i \quad b = b + \eta(y - y^{\wedge})$$

where η is the learning rate.

5. **Data/preprocessing problems:** Incorrect labels, inconsistent feature scaling, or wrongly formatted input data can prevent proper learning.

Debugging strategy:

- Verify that the dataset is actually linearly separable.
- Check the feature values and target labels for errors.
- Print the initial weights, bias, prediction and error for each training sample.
- Manually verify a few weight updates using the perceptron equation.
- Test the algorithm on a simple dataset such as **AND**.
- Check whether the loss/error decreases after each epoch.
- Confirm that the bias is included and updated correctly.

Optimization techniques:

- Select a suitable learning rate, such as 0.01 or 0.1.
- Normalize or standardize features when their scales differ significantly.
- Use deterministic initialization while debugging.
- Stop training when an epoch produces zero classification errors.
- Shuffle training samples between epochs if appropriate.

Thus, by checking the **data** → **prediction** → **error** → **update** → **convergence** pipeline, the implementation error can be isolated and corrected, leading to faster and stable convergence. This directly addresses the question's requested causes, debugging process, and optimization methods.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

(5 Marks)

Sol. A multilayer perceptron trained using backpropagation can converge slowly because of inappropriate activation functions, poor initialization, unsuitable learning rates, or vanishing gradients.

Possible causes:

1. **Vanishing gradients:** In deep networks, gradients can become extremely small during backpropagation, making earlier layers learn very slowly.
2. **Inappropriate activation functions:** Sigmoid or tanh may saturate for large positive or negative inputs, producing very small gradients.
3. **Poor weight initialization:** Very large or very small initial weights can make learning unstable or slow.
4. **Suboptimal learning rate:** A very small learning rate results in slow learning, while a very large rate may cause oscillation around the minimum.
5. **Unnormalized input:** Features with very different scales can make gradient-based optimization inefficient.

Step-by-step debugging:

1. Check the training and validation loss curves.
2. Verify whether the loss is actually decreasing.
3. Inspect the activations and gradients of each layer.
4. Check for extremely small or extremely large gradient values.
5. Verify the activation functions used in hidden layers.
6. Test different learning rates.
7. Check whether the input features are normalized.
8. Verify weight initialization and backpropagation calculations.

Optimization techniques:

- Normalize or standardize input features.
- Use **ReLU** or related activation functions in hidden layers to reduce vanishing-gradient problems.
- Use **Xavier/He initialization** for appropriate weight scaling.
- Use adaptive optimizers such as **Adam**.
- Apply learning-rate scheduling to reduce the learning rate as training progresses.

- Use mini-batch training for efficient gradient updates.

Therefore, monitoring **loss, gradients and activations layer-by-layer** helps identify the source of slow convergence, while normalization, suitable activation functions, proper initialization and adaptive optimizers improve training performance.

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

Sol. A genetic algorithm may produce different results in different runs because its population is initialized randomly and evolutionary operations depend on randomness. However, consistently poor solutions indicate problems in encoding, fitness evaluation or population diversity.

Possible causes:

1. **Improper chromosome encoding:** The chromosome may not correctly represent the possible hypotheses in the search space.
2. **Incorrect fitness function:** If fitness does not accurately represent solution quality, the algorithm may select poor solutions.
3. **Premature convergence:** The population may become too similar before finding the optimal solution.
4. **Lack of diversity:** Insufficient mutation or excessive selection pressure can reduce variation.
5. **Incorrect genetic operations:** Errors in crossover or mutation can generate invalid chromosomes.

Systematic debugging:

- Verify that every chromosome represents a valid hypothesis.
- Test the fitness function manually using known solutions.
- Print the best, worst and average fitness in every generation.
- Monitor the diversity of chromosomes.
- Check whether mutation and crossover produce valid offspring.
- Run the algorithm with a fixed random seed to reproduce errors.
- Compare results across several independent runs.

Optimization strategies:

- Adjust the **mutation rate** to maintain population diversity.
- Reduce excessive selection pressure so weaker but potentially useful solutions are not eliminated too quickly.
- Use **elitism** to preserve the best individuals between generations.

- Select an appropriate population size.
- Use suitable crossover operators for the chromosome representation.
- Maintain a balance between **exploration** of new solutions and **exploitation** of good existing solutions.

For example, if all chromosomes become nearly identical after only a few generations, mutation can be increased and selection pressure reduced. This helps prevent premature convergence while maintaining good solutions through elitism.

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

(5 Marks)

A deep neural network combined with genetic programming can have a large number of parameters and candidate solutions. This increases computational cost and can cause the model to memorize training data instead of generalizing to unseen data.

Possible causes:

1. **Excessive model complexity:** Too many layers, neurons or genetic-programming structures may not be necessary.
2. **Over-parameterization:** A large number of parameters increases the ability of the model to fit noise.
3. **Insufficient training data:** A complex model trained using limited data is more likely to overfit.
4. **Expensive genetic evaluation:** Evaluating a large population of complex candidate programs requires considerable computational resources.

Debugging steps:

- Compare training and validation performance.
- If training accuracy is high but validation accuracy is low, investigate overfitting.
- Measure the number of network parameters.
- Examine the complexity and depth of generated genetic programs.
- Check whether the training dataset is sufficiently representative.
- Measure computation time for each generation/evaluation.

Optimization techniques:

- **Regularization:** Add penalties such as L1/L2 regularization to discourage overly complex models.
- **Dropout:** Randomly deactivate neurons during training to reduce dependency on particular neurons.

- **Pruning:** Remove unnecessary network connections, neurons or genetic-programming branches.
- **Early stopping:** Stop training when validation performance stops improving.
- **Efficient evaluation:** Evaluate simpler candidate programs first and avoid repeatedly evaluating unnecessarily complex structures.
- **Increase training data:** More representative data can improve generalization.

The goal is not simply to maximize training accuracy. A good solution should achieve a balance between **accuracy, model complexity, computational cost and generalization.**

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5Marks)

This situation is a typical indication of **overfitting**. The decision tree has learned the training data too specifically, including noise or unusual patterns, instead of learning general relationships.

Possible causes:

1. **Overfitting:** The tree memorizes training examples.
2. **Excessive depth:** A very deep tree creates many specific branches and leaf nodes.
3. **Noisy data:** Noise and outliers may cause unnecessary splits.
4. **Too many features:** Irrelevant features may produce misleading splits.
5. **Insufficient validation:** The model may have been evaluated only on the training data.

Debugging steps:

- Compare training accuracy with validation/test accuracy.
- Inspect the depth and number of nodes in the tree.
- Identify branches containing very few training samples.
- Check the dataset for noisy or irrelevant features.
- Use cross-validation to determine whether the model generalizes consistently.
- Compare performance before and after limiting tree depth.

Optimization methods:

1. **Pruning:** Remove branches that contribute little to predictive performance.
2. **Limit tree depth:** Set a maximum depth to prevent excessive complexity.
3. **Minimum samples per leaf:** Prevent leaves from being created from very few samples.
4. **Cross-validation:** Test different hyperparameter combinations and select the model that generalizes best.
5. **Feature selection:** Remove irrelevant or redundant features.

6. **Use validation data:** Select the final model based on unseen validation performance rather than training accuracy.

For example, if a tree has **99% training accuracy but only 70% validation accuracy**, the large gap indicates overfitting. Reducing tree depth and applying pruning can produce a simpler tree with slightly lower training accuracy but substantially better performance on unseen data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand